```
--
============================================
================================
-- 0001_init.sql — Academic Platform V1
schema
-- Level → Semester → Course → Content
Item → Content Material
-- Includes: constraints, indexes, RLS,
private lookup, storage policies.
--
============================================
================================

begin;

create extension if not exists pgcrypto;

-- ----------------------------------------
----------------------------------
-- Helpers
-- ----------------------------------------
----------------------------------

create or replace function
public.set_updated_at()
returns trigger
language plpgsql
as $$
begin
  new.updated_at = now();
```

```
  return new;
end;
$$;

-- ------------------------------------
-----------------------------------
-- levels
-- ------------------------------------
-----------------------------------

create table public.levels (
  id          uuid primary key default
gen_random_uuid(),
  name        text not null check
(length(trim(name)) between 1 and 120),
  order_index integer not null,
  created_at  timestamptz not null default
now(),
  updated_at  timestamptz not null default
now(),
  constraint levels_order_uq unique
(order_index) deferrable initially deferred
);

create unique index levels_name_uq on
public.levels (lower(name));

create trigger levels_set_updated_at before
update on public.levels
  for each row execute function
public.set_updated_at();

-- ------------------------------------
-----------------------------------
```

```
-- semesters
-- ---------------------------------------
----------------------------------

create table public.semesters (
  id          uuid primary key default
gen_random_uuid(),
  level_id    uuid not null references
public.levels(id) on delete restrict,
  name        text not null check
(length(trim(name)) between 1 and 120),
  order_index integer not null,
  created_at  timestamptz not null default
now(),
  updated_at  timestamptz not null default
now(),
  constraint semesters_order_uq unique
(level_id, order_index) deferrable
initially deferred
);

create unique index semesters_name_uq on
public.semesters (level_id, lower(name));
-- Required as target of the profiles
(semester_id, level_id) foreign key:
create unique index semesters_id_level_uq
on public.semesters (id, level_id);

create trigger semesters_set_updated_at
before update on public.semesters
  for each row execute function
public.set_updated_at();

-- ---------------------------------------
```

```
-----------------------------------
-- courses
-- ----------------------------------------
-----------------------------------

create table public.courses (
  id          uuid primary key default
gen_random_uuid(),
  semester_id uuid not null references
public.semesters(id) on delete restrict,
  name        text not null check
(length(trim(name)) between 1 and 200),
  code        text not null check (code ~
'^[A-Za-z0-9_-]{1,32}$'),
  description text not null default '',
  order_index integer not null,
  status      text not null default 'draft'
              check (status in ('draft',
'published', 'archived')),
  created_at  timestamptz not null default
now(),
  updated_at  timestamptz not null default
now(),
  constraint courses_order_uq unique
(semester_id, order_index) deferrable
initially deferred
);

create unique index courses_code_uq on
public.courses (semester_id, lower(code));
create index courses_semester_order_idx on
public.courses (semester_id, order_index);

create trigger courses_set_updated_at
```

```
before update on public.courses
  for each row execute function
public.set_updated_at();

-- ----------------------------------------
-----------------------------------
-- content_items
-- ----------------------------------------
-----------------------------------

create table public.content_items (
  id          uuid primary key default
gen_random_uuid(),
  course_id   uuid not null references
public.courses(id) on delete restrict,
  title       text not null check
(length(trim(title)) between 1 and 200),
  description text not null default '',
  order_index integer not null,
  status      text not null default 'draft'
              check (status in ('draft',
'published', 'archived')),
  created_at  timestamptz not null default
now(),
  updated_at  timestamptz not null default
now(),
  constraint content_items_order_uq unique
(course_id, order_index) deferrable
initially deferred
);

create index content_items_course_order_idx
on public.content_items (course_id,
order_index);
```

```
create trigger content_items_set_updated_at
before update on public.content_items
  for each row execute function
public.set_updated_at();

-- ----------------------------------------
---------------------------------
-- material_kinds
-- ----------------------------------------
---------------------------------

create table public.material_kinds (
  id          uuid primary key default
gen_random_uuid(),
  code        text not null unique check
(code ~ '^[a-z0-9_]{2,64}$'),
  name        text not null,
  description text not null default '',
  created_at  timestamptz not null default
now()
);

-- ----------------------------------------
---------------------------------
-- profiles
-- ----------------------------------------
---------------------------------

create table public.profiles (
  id           uuid primary key references
auth.users(id) on delete cascade,
  full_name    text not null check
(length(trim(full_name)) between 2 and
```

```
200),
  university_id text,
  role         text not null default
'student' check (role in ('student',
'admin')),
  level_id     uuid references
public.levels(id) on delete restrict,
  semester_id  uuid references
public.semesters(id) on delete restrict,
  is_active    boolean not null default
true,
  created_by   uuid references
public.profiles(id) on delete set null,
  created_at   timestamptz not null default
now(),
  updated_at   timestamptz not null default
now(),
  -- Students must be fully assigned;
admins have no academic assignment.
  constraint profiles_assignment_chk check
(
    (role = 'admin' and level_id is null
and semester_id is null)
    or (role = 'student' and university_id
is not null
        and length(university_id) between 2
and 64 and level_id is not null and
semester_id is not null)
  ),
  -- Semester must actually belong to the
assigned level.
  foreign key (semester_id, level_id)
    references public.semesters (id,
level_id) on delete restrict
```

```
);

-- One active university number per student
(inactive/replaced numbers may be reused).
create unique index
profiles_university_active_uq
  on public.profiles (university_id)
  where role = 'student' and is_active =
true;

create trigger profiles_set_updated_at
before update on public.profiles
  for each row execute function
public.set_updated_at();

-- Block privilege escalation / academic
reassignment by non-admins, even if
-- a client crafts an UPDATE directly
against the table.
create or replace function
public.profiles_guard_update()
returns trigger
language plpgsql
as $$
begin
  if not public.is_admin() then
    if new.role is distinct from old.role
       or new.university_id is distinct
from old.university_id
       or new.level_id is distinct from
old.level_id
       or new.semester_id is distinct from
old.semester_id
       or new.is_active is distinct from
```

```
old.is_active then
      raise exception 'not permitted';
    end if;
  end if;
  return new;
end;
$$;

create trigger profiles_guard_update_trg
  before update on public.profiles
  for each row execute function
public.profiles_guard_update();

-- SECURITY DEFINER helpers: these run as
the function owner (postgres), so
-- they can read profiles inside RLS
policies without circular recursion.
-- search_path is pinned to '' so no
attacker-controlled schema hijacking.

create or replace function
public.current_profile()
returns public.profiles
language sql stable
security definer
set search_path = ''
as $$
  select * from public.profiles where id =
auth.uid();
$$;

create or replace function
public.is_admin()
returns boolean
```

```
language sql stable
security definer
set search_path = ''
as $$
  select exists (
    select 1 from public.profiles
    where id = auth.uid() and role =
'admin' and is_active = true
  );
$$;

create or replace function
public.is_active_student()
returns boolean
language sql stable
security definer
set search_path = ''
as $$
  select exists (
    select 1 from public.profiles
    where id = auth.uid() and role =
'student' and is_active = true
      and level_id is not null and
semester_id is not null
  );
$$;

create or replace function
public.my_semester_id()
returns uuid
language sql stable
security definer
set search_path = ''
as $$
```

```
  select semester_id from public.profiles
where id = auth.uid();
$$;

create or replace function
public.my_level_id()
returns uuid
language sql stable
security definer
set search_path = ''
as $$
  select level_id from public.profiles
where id = auth.uid();
$$;

revoke all on function
public.current_profile() from public, anon;
revoke all on function public.is_admin()
from public, anon;
revoke all on function
public.is_active_student() from public,
anon;
revoke all on function
public.my_semester_id() from public, anon;
revoke all on function public.my_level_id()
from public, anon;
grant execute on function
public.current_profile() to authenticated;
grant execute on function public.is_admin()
to authenticated;
grant execute on function
public.is_active_student() to
authenticated;
grant execute on function
```

```
public.my_semester_id() to authenticated;
grant execute on function
public.my_level_id() to authenticated;

-- ---------------------------------------
----------------------------------
-- content_materials
-- Articles are stored as validated JSONB
(schemaVersion inside the document,
-- validated again with Zod at the
application layer before storage).
-- The original source file always remains
available.
-- ---------------------------------------
----------------------------------

create table public.content_materials (
  id              uuid primary key default
gen_random_uuid(),
  content_item_id uuid not null references
public.content_items(id) on delete
restrict,
  material_kind_id uuid not null references
public.material_kinds(id) on delete
restrict,
  title           text not null check
(length(trim(title)) between 1 and 200),
  description     text not null default '',
  storage_path    text,
  status          text not null default
'draft'
                  check (status in
('draft', 'published', 'archived')),
  order_index     integer not null,
```

```
  created_by      uuid references
public.profiles(id) on delete set null,
  created_at      timestamptz not null
default now(),
  updated_at      timestamptz not null
default now(),
  article         jsonb,
  constraint content_materials_order_uq
unique (content_item_id, order_index)
deferrable initially deferred,
  constraint
content_materials_file_or_article_chk
    check (storage_path is not null or
article is not null)
);

create index
content_materials_item_order_idx on
public.content_materials (content_item_id,
order_index);

create trigger
content_materials_set_updated_at before
update on public.content_materials
  for each row execute function
public.set_updated_at();

-- ----------------------------------------
-----------------------------------
-- Reordering (single-transaction neighbor
swap; deferred unique constraints
-- make the two UPDATEs safe inside one
function call)
-- ----------------------------------------
```

```
-----------------------------------

create or replace function
public.move_item(p_kind text, p_id uuid,
p_delta integer)
returns void
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_order integer;
  v_neighbor uuid;
begin
  if not public.is_admin() then
    raise exception 'not permitted';
  end if;
  if p_delta <> 1 and p_delta <> -1 then
    raise exception 'invalid delta';
  end if;

  if p_kind = 'level' then
    select order_index into v_order from
public.levels where id = p_id;
    if v_order is null then raise exception
'not found'; end if;
    select id into v_neighbor from
public.levels
      where order_index = v_order + p_delta
order by order_index limit 1;
    if v_neighbor is null then return; end
if;
    update public.levels set order_index =
order_index + p_delta where id = p_id;
```

```
    update public.levels set order_index =
order_index - p_delta where id =
v_neighbor;
  elsif p_kind = 'semester' then
    select order_index into v_order from
public.semesters where id = p_id;
    if v_order is null then raise exception
'not found'; end if;
    select id into v_neighbor from
public.semesters
      where level_id = (select level_id
from public.semesters where id = p_id)
        and order_index = v_order + p_delta
limit 1;
    if v_neighbor is null then return; end
if;
    update public.semesters set order_index
= order_index + p_delta where id = p_id;
    update public.semesters set order_index
= order_index - p_delta where id =
v_neighbor;
  elsif p_kind = 'course' then
    select order_index into v_order from
public.courses where id = p_id;
    if v_order is null then raise exception
'not found'; end if;
    select id into v_neighbor from
public.courses
      where semester_id = (select
semester_id from public.courses where id =
p_id)
        and order_index = v_order + p_delta
limit 1;
    if v_neighbor is null then return; end
```

```
if;
    update public.courses set order_index =
order_index + p_delta where id = p_id;
    update public.courses set order_index =
order_index - p_delta where id =
v_neighbor;
  elsif p_kind = 'content_item' then
    select order_index into v_order from
public.content_items where id = p_id;
    if v_order is null then raise exception
'not found'; end if;
    select id into v_neighbor from
public.content_items
      where course_id = (select course_id
from public.content_items where id = p_id)
        and order_index = v_order + p_delta
limit 1;
    if v_neighbor is null then return; end
if;
    update public.content_items set
order_index = order_index + p_delta where
id = p_id;
    update public.content_items set
order_index = order_index - p_delta where
id = v_neighbor;
  elsif p_kind = 'material' then
    select order_index into v_order from
public.content_materials where id = p_id;
    if v_order is null then raise exception
'not found'; end if;
    select id into v_neighbor from
public.content_materials
      where content_item_id = (select
content_item_id from
```

```
public.content_materials where id = p_id)
        and order_index = v_order + p_delta
limit 1;
    if v_neighbor is null then return; end
if;
    update public.content_materials set
order_index = order_index + p_delta where
id = p_id;
    update public.content_materials set
order_index = order_index - p_delta where
id = v_neighbor;
  else
    raise exception 'unknown kind';
  end if;
end;
$$;

revoke all on function
public.move_item(text, uuid, integer) from
public, anon;
grant execute on function
public.move_item(text, uuid, integer) to
authenticated;

-- ----------------------------------------
-----------------------------------
-- RLS
-- ----------------------------------------
-----------------------------------

alter table public.levels enable row level
security;
alter table public.semesters enable row
level security;
```

```
alter table public.courses enable row level
security;
alter table public.content_items enable row
level security;
alter table public.content_materials enable
row level security;
alter table public.profiles enable row
level security;
alter table public.material_kinds enable
row level security;

-- profiles
create policy profiles_select on
public.profiles for select to authenticated
  using (id = auth.uid() or
public.is_admin());
create policy profiles_insert on
public.profiles for insert to authenticated
  with check (public.is_admin());
create policy profiles_update on
public.profiles for update to authenticated
  using (id = auth.uid() or
public.is_admin())
  with check (id = auth.uid() or
public.is_admin());

-- levels: active students read; admins
manage
create policy levels_select on
public.levels for select to authenticated
  using (public.is_admin() or
public.is_active_student());
create policy levels_insert on
public.levels for insert to authenticated
```

```
  with check (public.is_admin());
create policy levels_update on
public.levels for update to authenticated
  using (public.is_admin()) with check
(public.is_admin());

-- semesters
create policy semesters_select on
public.semesters for select to
authenticated
  using (public.is_admin() or id =
public.my_semester_id());
create policy semesters_insert on
public.semesters for insert to
authenticated
  with check (public.is_admin());
create policy semesters_update on
public.semesters for update to
authenticated
  using (public.is_admin()) with check
(public.is_admin());

-- courses (students: only published
courses of their own semester)
create policy courses_select on
public.courses for select to authenticated
  using (
    public.is_admin()
    or (status = 'published' and
semester_id = public.my_semester_id())
  );
create policy courses_insert on
public.courses for insert to authenticated
  with check (public.is_admin());
```

```
create policy courses_update on
public.courses for update to authenticated
  using (public.is_admin()) with check
(public.is_admin());

-- content_items
create policy content_items_select on
public.content_items for select to
authenticated
  using (
    public.is_admin()
    or (status = 'published' and exists (
      select 1 from public.courses c
      where c.id = content_items.course_id
        and c.status = 'published'
        and c.semester_id =
public.my_semester_id()
    ))
  );
create policy content_items_insert on
public.content_items for insert to
authenticated
  with check (public.is_admin());
create policy content_items_update on
public.content_items for update to
authenticated
  using (public.is_admin()) with check
(public.is_admin());

-- content_materials
create policy content_materials_select on
public.content_materials for select to
authenticated
  using (
```

```
    public.is_admin()
    or (status = 'published' and exists (
      select 1
      from public.content_items i
      join public.courses c on c.id =
i.course_id
      where i.id =
content_materials.content_item_id
        and i.status = 'published'
        and c.status = 'published'
        and c.semester_id =
public.my_semester_id()
    ))
  );
create policy content_materials_insert on
public.content_materials for insert to
authenticated
  with check (public.is_admin());
create policy content_materials_update on
public.content_materials for update to
authenticated
  using (public.is_admin()) with check
(public.is_admin());

-- material_kinds
create policy material_kinds_select on
public.material_kinds for select to
authenticated
  using (true);
create policy material_kinds_insert on
public.material_kinds for insert to
authenticated
  with check (public.is_admin());
```

```
-- ------------------------------------
-----------------------------------
-- Restricted schema: private university-
number → internal auth email lookup.
-- EXECUTE is granted ONLY to service_role;
the browser never touches this.
-- ------------------------------------
-----------------------------------

create schema if not exists app_private;
revoke all on schema app_private from
public, anon, authenticated;

create or replace function
app_private.student_auth_email(p_university
_id text)
returns text
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_email text;
begin
  if p_university_id is null or
length(trim(p_university_id)) < 2 then
    return null;
  end if;
  select u.email into v_email
  from public.profiles p
  join auth.users u on u.id = p.id
  where p.role = 'student'
    and p.is_active = true
    and p.university_id = p_university_id;
```

```
  return v_email; -- null when
missing/inactive => generic login error
upstream
end;
$$;

revoke all on function
app_private.student_auth_email(text) from
public, anon, authenticated;
grant execute on function
app_private.student_auth_email(text) to
service_role;

-- ----------------------------------------
----------------------------------
-- Private Storage bucket + policies
-- Path format:
{course_id}/{content_item_id}/{material_id}
.{ext}
-- ----------------------------------------
----------------------------------

insert into storage.buckets (id, name,
public)
values ('content-files', 'content-files',
false)
on conflict (id) do nothing;

-- Is the caller's student profile allowed
to read this object path?
create or replace function
app_private.path_authorized_for_student(p_p
ath text)
returns boolean
```

```
language plpgsql
stable
security definer
set search_path = ''
as $$
declare
  v_parts text[];
begin
  v_parts := string_to_array(p_path, '/');
  if v_parts is null or
array_length(v_parts, 1) <> 3 then
    return false;
  end if;
  return exists (
    select 1
    from public.content_materials m
    join public.content_items i on i.id =
m.content_item_id
    join public.courses c on c.id =
i.course_id
    where c.id::text = v_parts[1]
      and i.id::text = v_parts[2]
      and m.id::text =
split_part(v_parts[3], '.', 1)
      and m.status = 'published'
      and i.status = 'published'
      and c.status = 'published'
      and c.semester_id =
public.my_semester_id()
  );
end;
$$;

revoke all on function
```

```
app_private.path_authorized_for_student(tex
t) from public, anon;
grant execute on function
app_private.path_authorized_for_student(tex
t) to authenticated;

create policy content_files_select on
storage.objects for select to authenticated
  using (
    bucket_id = 'content-files'
    and (public.is_admin() or
app_private.path_authorized_for_student(nam
e))
  );

create policy content_files_insert on
storage.objects for insert to authenticated
  with check (bucket_id = 'content-files'
and public.is_admin());

create policy content_files_update on
storage.objects for update to authenticated
  using (bucket_id = 'content-files' and
public.is_admin())
  with check (bucket_id = 'content-files'
and public.is_admin());

create policy content_files_delete on
storage.objects for delete to authenticated
  using (bucket_id = 'content-files' and
public.is_admin());

commit;
```